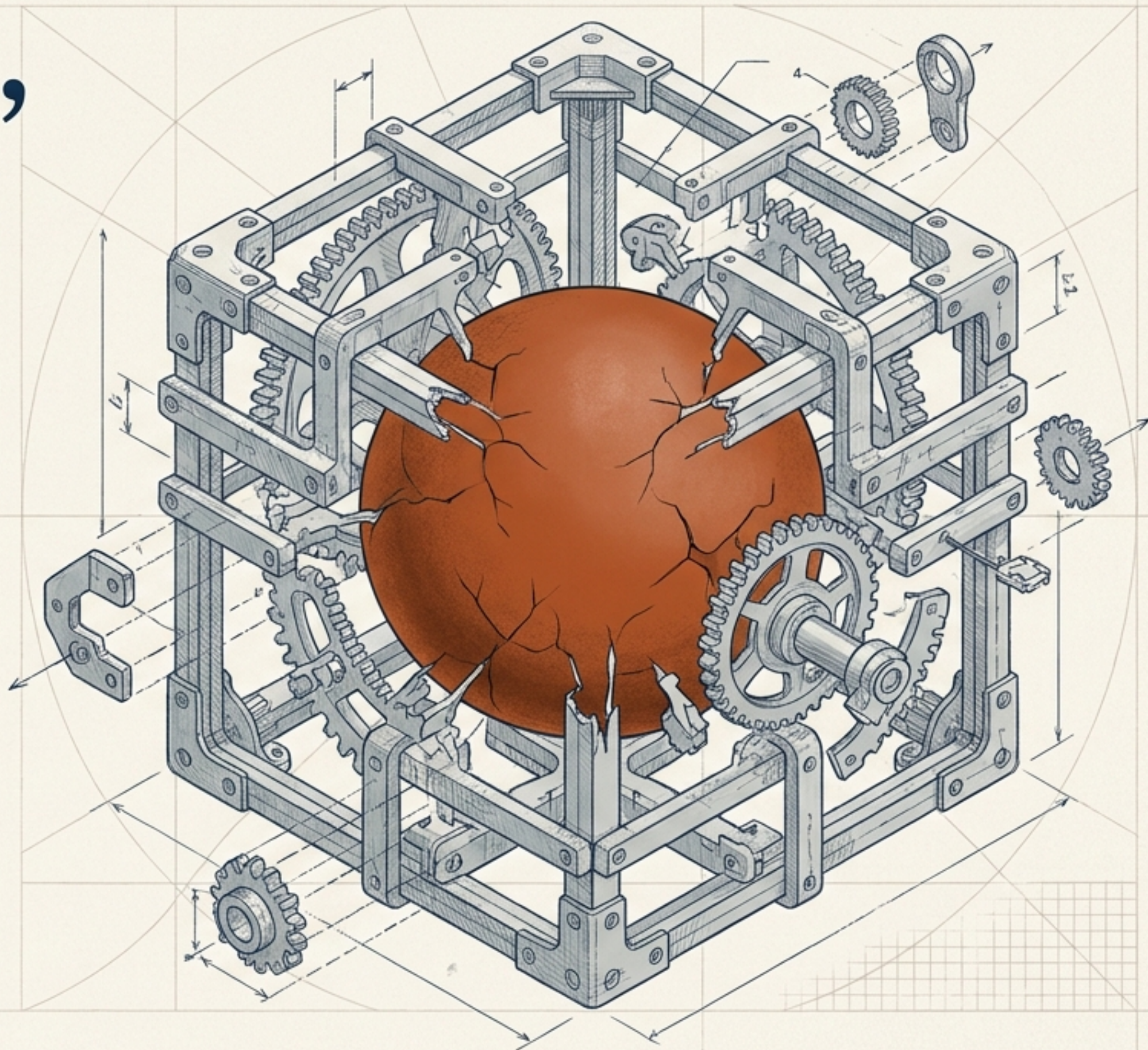


# 你的舊提示詞， 正在拖垮新一 一代模型

Mythos 與新一代 AI 降臨前的  
系統重塑指南。

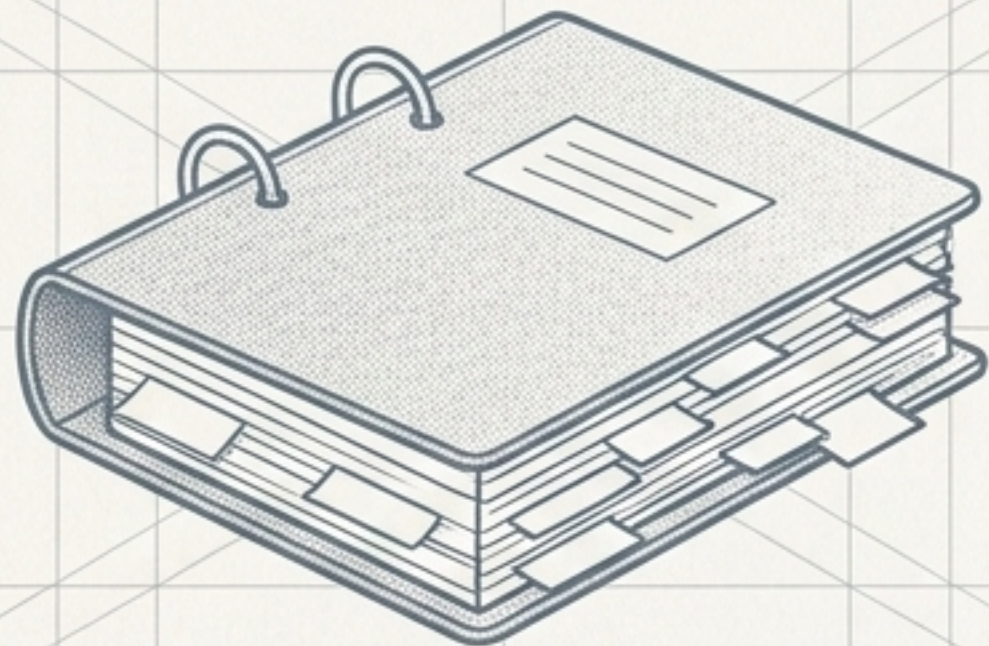
捨棄保姆式干預，擁抱自主性架構。



# 模型的成長速度，已經超越了系統設計的迭代速度



# 從管理「實習生」到賦權「準主管」



三年前的模型（實習生級別）

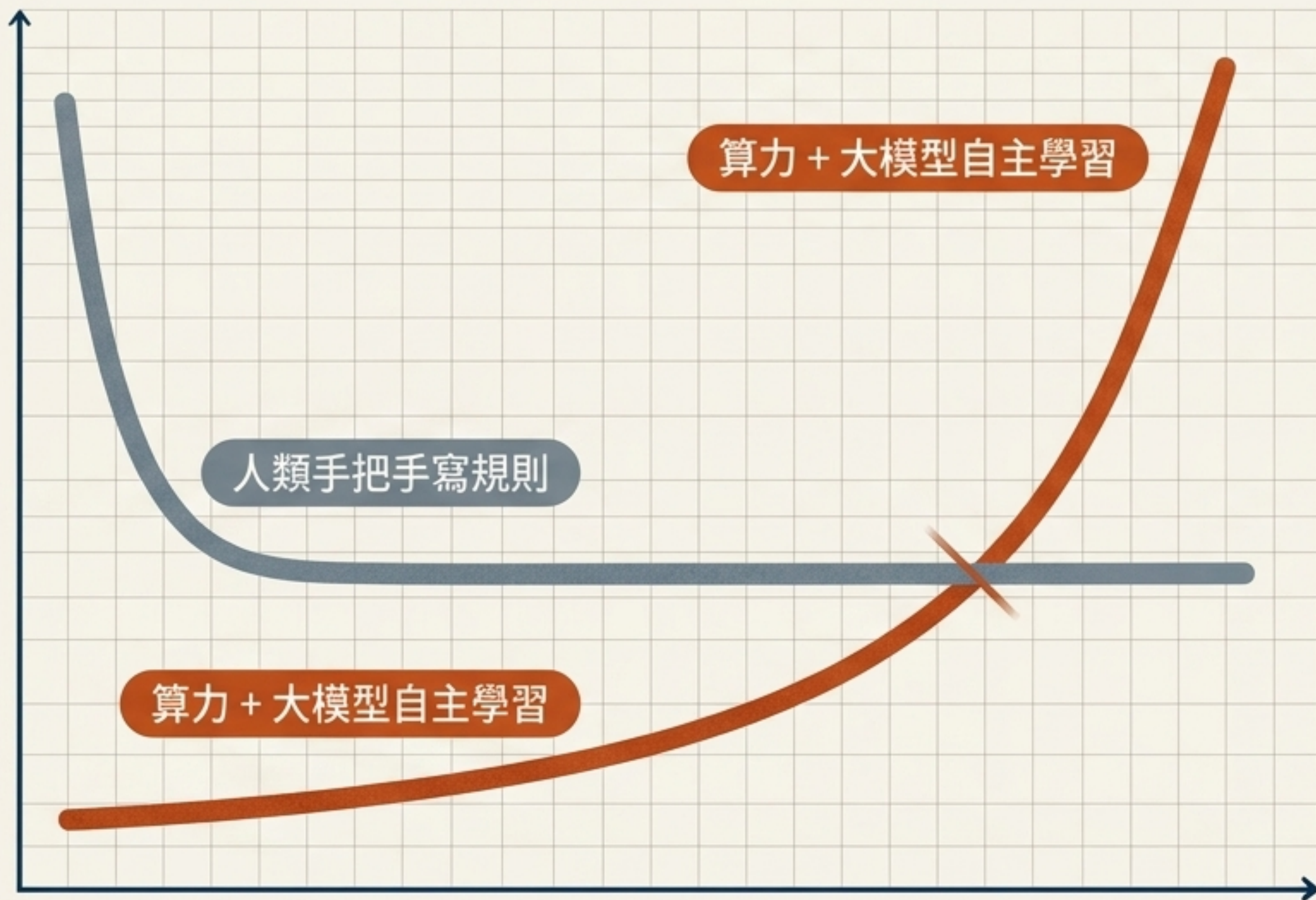
|      |                       |
|------|-----------------------|
| 你需要給 | 詳盡的新人手冊。              |
| 溝通方式 | 第一步做 A，第二步做 B，最後檢查 C。 |
| 預期結果 | 照本宣科，避免出錯。            |



現在的模型（準主管級別）

|      |                             |
|------|-----------------------------|
| 你需要給 | 明確的團隊目標。                    |
| 溝通方式 | 給予目標與資源，讓他自己 Figure it out。 |
| 預期結果 | 靈活應變，找出比你更好的路徑。             |

# AI 發展史反覆驗證的鐵則：The Bitter Lesson



每當人類想靠自己的專業知識，手把手去教 AI 怎麼做事，最後都會輸給丟入更多算力、讓機器自主學習的方法。

— Rich Sutton, 2019

你塞進 System Prompt 的每一個 How-to，都是在賭模型不會變聰明。當模型進化，你的人為干預就會從「助力」變成「阻力」。

# 捨不得刪除複雜代碼，往往是源於開發者的「身份焦慮」



## 迷思 (The Myth)

我們將「長達 3000 Tokens 的系統指令」、「客製化的 RAG Pipeline」與「精密的 Chain of Thought 框架」視為自身 AI 技術能力的證明。

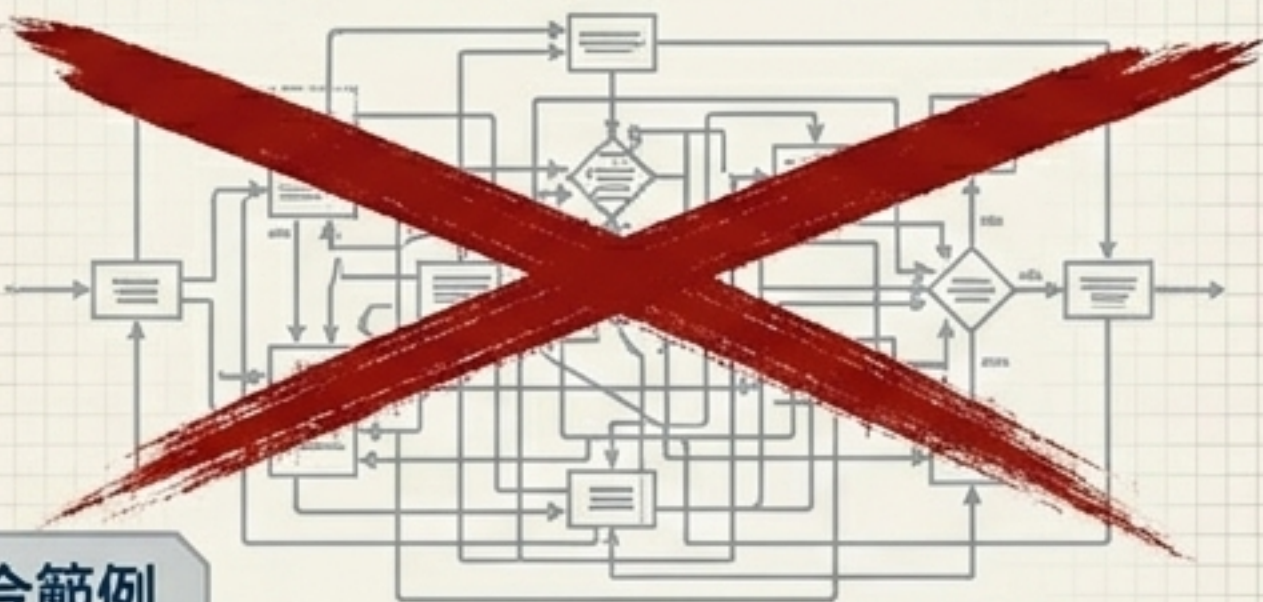
## 真相 (The Truth)

這就像捨不得孩子長大的父母。享受「被需要」的感覺，反而阻礙了新模型的潛力發揮。

Anthropic 與 OpenAI 官方一致建議：停止冗長的風格規定與程式碼要求，讓 Intelligent 的模型去做 Intelligent 的事。

# 必須捨棄的包袱一：保姆式提示詞 (Nanny-Style Prompts)

## 舊思維：指揮新手司機



### 指令範例

先分類意圖為 14 類之一 -> 撈前五篇文章 -> 只能用查到的 Context -> 檢查 URL 是否幻覺。

問題：模型被寫死的路徑綁架，無法發揮推論能力。

## 新思維：信任老司機



### 指令範例

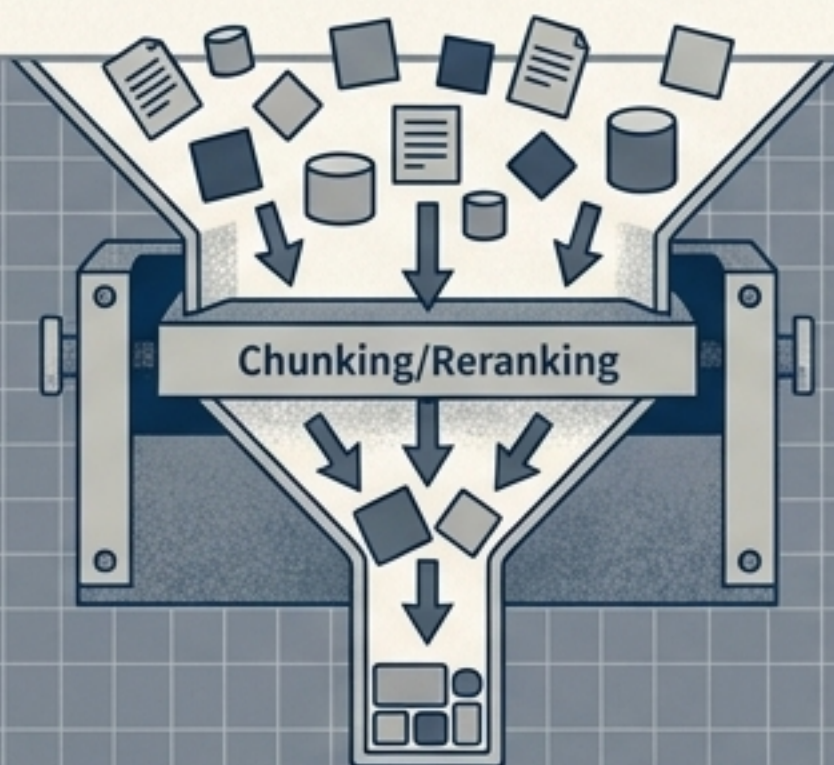
解決客戶問題，確保客戶滿意離開，且處理方式符合公司退換貨政策。

優勢：只給終點與邊界。老司機自己知道哪條路最順、如何避開塞車。

一秒自我檢測：如果把指令拿掉，產出品質沒變甚至更好，就勇敢刪除！（實測：100行指令縮減至50行，輸出反而更有料）。

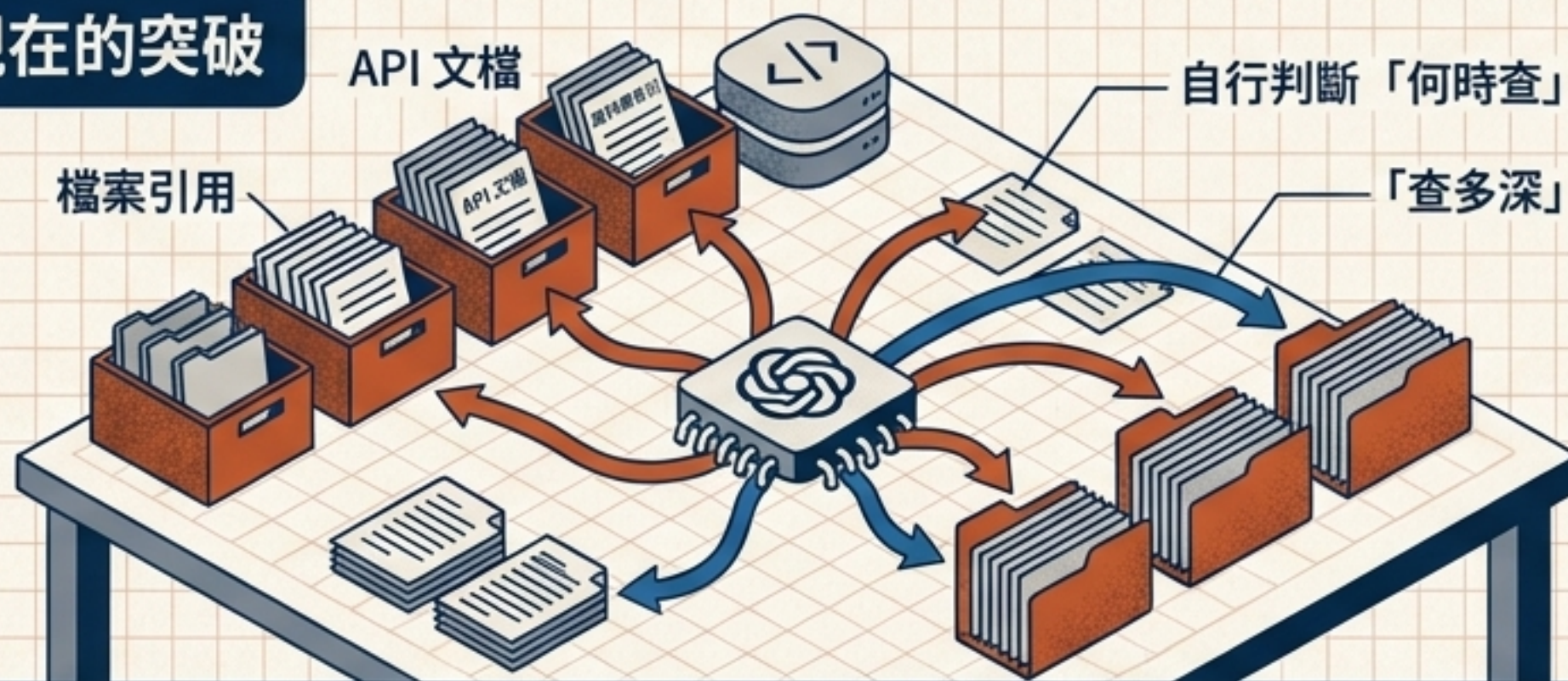
# 必須捨棄的包袱二：死板的檢索框架 (Rigid Retrieval)

## 過去的限制



Context Window 小，模型不會自己查。  
人類必須預先猜測模型需要什麼，排好序  
塞進去。

## 現在的突破



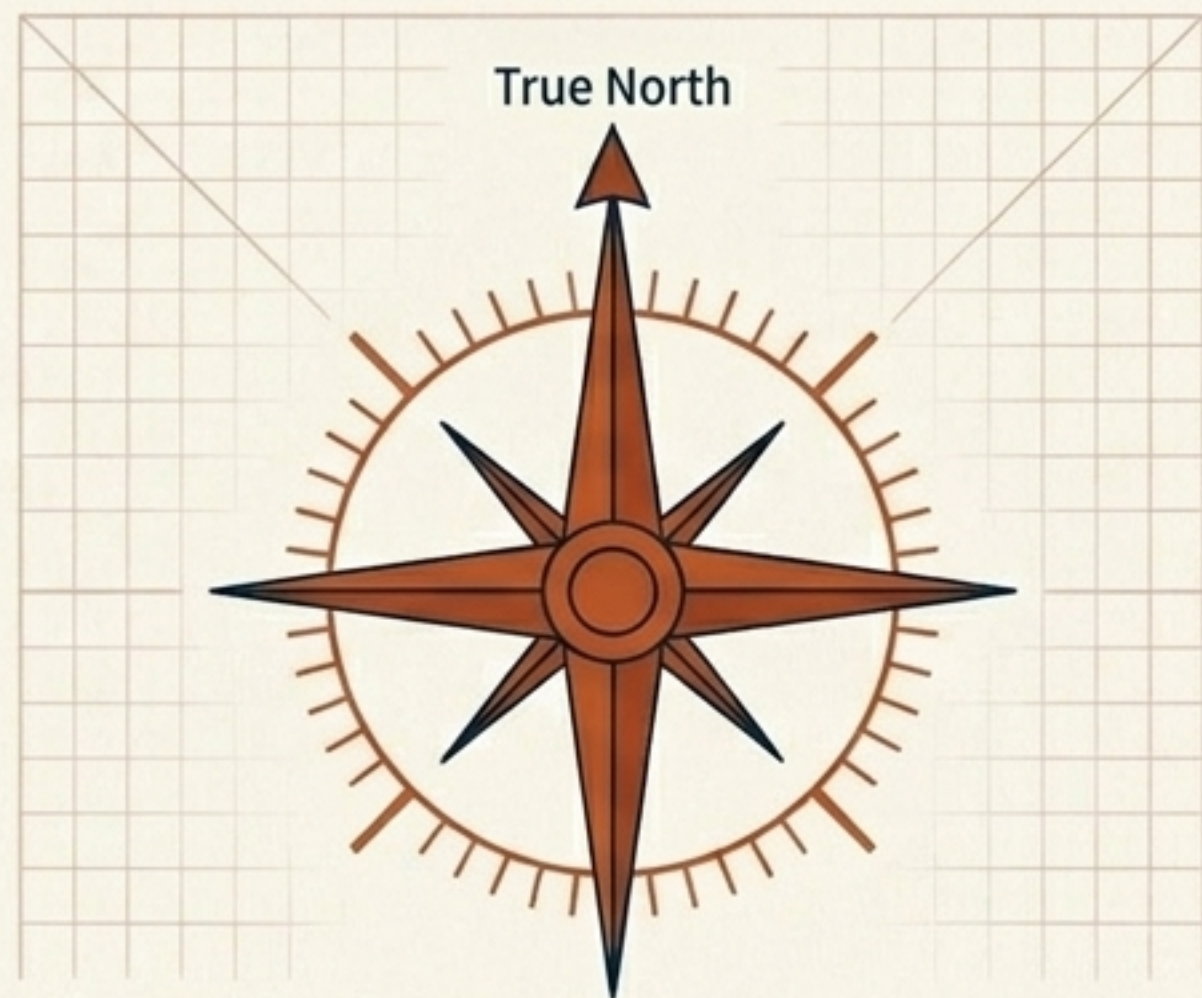
百萬 Token 時代來臨。前期只給輕量檔案引用，讓模型自行判斷「何時查」與「查多深」。  
(以 Anthropic Claude Code 為例)

**權力轉移不等於擺爛！** 模型變聰明的是搜尋邏輯，但資料存哪、權限控管、Token 預算限制的基礎建設，依然是工程師的職責。

# 必須捨棄的包袱三：寫死的規則 (Hard-Coded Rules)

## 脆弱的決策樹

「如果小於 30 天，且非特價品，  
則退款，否則轉人工...」  
當 Prompt 塞滿 40-80 條  
IF-THEN 規則時，  
系統就失去了彈性。



## 商業世界的真相

你永遠無法窮舉所有的 Corner Cases。

**解法：**給予「最高指導原則」。強大的模型具備舉一反三的靈機應變能力，能妥善處理未定義的雜湊情境。

# 實戰診斷：這條規則該砍嗎？

核心提問：這條規則是為了符合法規，還是為了怕模型出錯？

Business Rule (業務規則)

特徵：法規要求、權限底線、退款上限。

判斷：不管未來模型多強大都必須遵守的人類界線。



保留 (Keep)

Model Rule (模型防呆規則)

特徵：檢查語氣、不可亂編產品名、不確定要說不知道。

判斷：如果拿掉這條，新模型自己也能做對嗎？如果能，這就是過期的防呆機制。



刪除 (Purge)

# 相容新世代模型的四大架構要素

## 1. Prompt Spec (目標規格)

指出終點在哪，以及如何定義成功，而不是給定路線。

## 2. Guardrails (護欄)

絕對不可打破的 Business Rules (資安、法規)。

## 3. Tool List (工具清單)

提供邊界清晰的工具 API，讓模型隨時調用。

## 4. Collaboration Mode (協作模式)

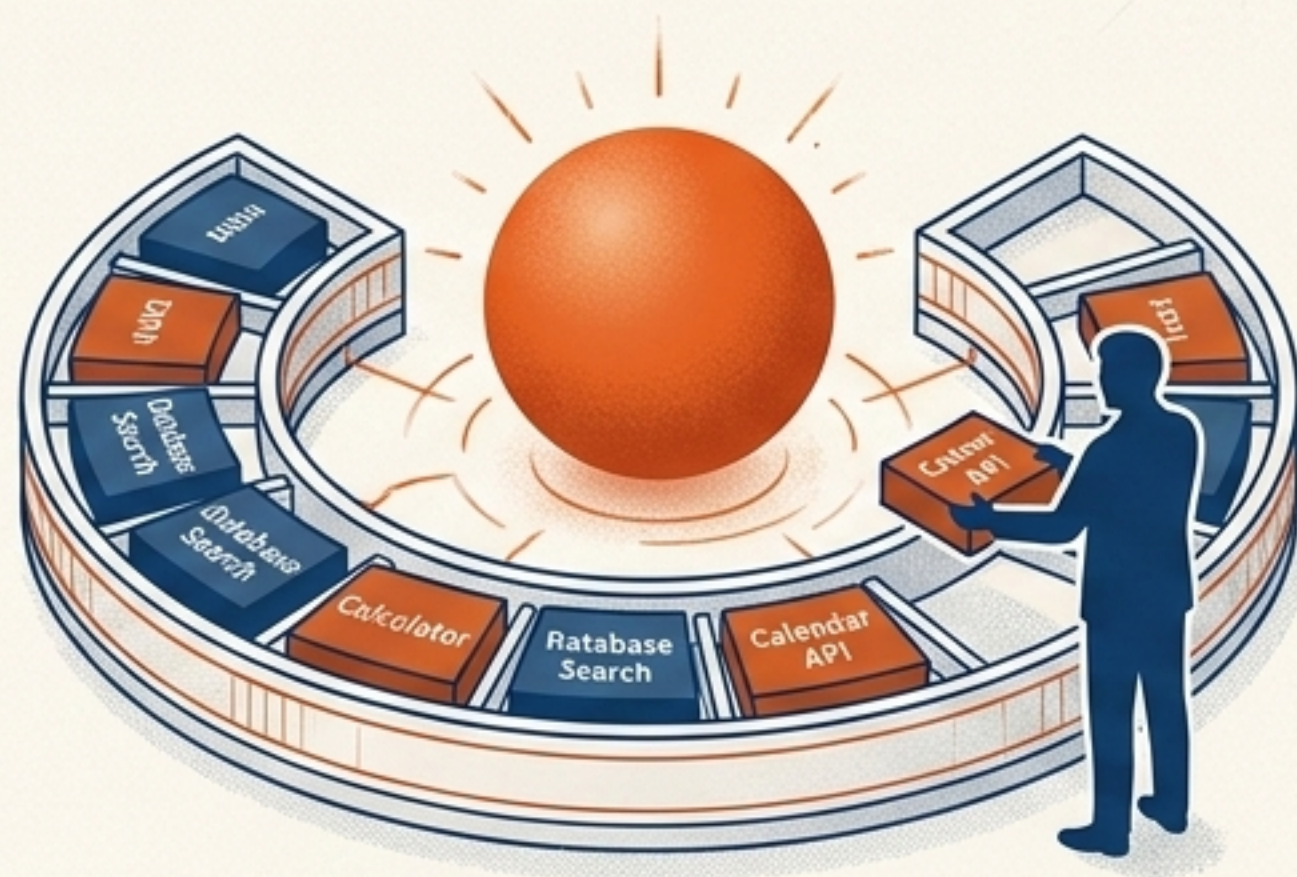
只定義「誰規劃、誰執行、誰評估」，不微操互動細節。

系統知道的「怎麼做 (How-to)」越少，就越能無痛吃下新模型升級帶來的紅利。

# 控制權轉移：從指揮家 (Orchestrator) 到軍火商 (Tool Provider)



舊架構：人類決定調用順序，寫 Prompt 告訴模型何時使用哪個工具。



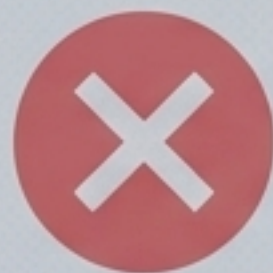
新架構：模型自己就是 Orchestrator。

## 新的挑戰

這不是降級，而是升級。準備工具比設計流程 更難！你的核心工作變成：想清楚每個工具的邊界、輸入/輸出格式、以及權限範圍。工具定義得越乾淨，模型用得越順手。

# 系統指令重塑：成果導向 vs 流程導向

流程導向  
(Process-Oriented)



容易被淘汰

寫法：

1. 先分意圖 → 2. 查知識庫 → 3. 生成回應 → 4. 檢查幻覺與連結。

致命傷：

把做法寫死。模型每次升級，整套系統都要重構。

成果導向  
(Outcome-Oriented)



無痛自動升級

寫法：

1. 解決客戶當下的問題



2. 確保客戶離開時是滿意的



3. 任何處理方式必須嚴格符合公司的退换货政策。

優勢：

指明成功的樣貌。模型越聰明，它自己想出來的達成路徑就越漂亮。

# 重新定義 AI Builder：2026 年的核心競爭力

逐漸貶值的技能 (被模型取代)

指數升值的技能 (通用工作能力放大)

精密的 Prompt 工程 (微調字眼)

1. 定義目標 (Setting Goals) - 能清晰描述系統「成功的樣貌」與「失敗的底線」。

設計複雜的 Chain of Thought

2. 提供脈絡 (Providing Context) - 精準判斷模型在當下需要什麼政策文件與背景資訊，才能做出正確決策。

手動微調 RAG pipeline 檢索權重

模型越強大，越需要的是清晰的方向與乾淨的資料，而不是冗長囉嗦的步驟。

# Mythos 降臨前的 30 分鐘行動計畫



## 檢視並刪除「保姆式提示詞」

測試拔掉步驟指令，確認模型產出是否依然穩定。



## 放權檢索流程

停止硬塞所有資料，嘗試給予模型搜尋工具與自主判斷權。



## 剝離「Model Rule」

砍掉防呆機制，只將「Business Rule (業務法規)」列入護欄。

模型升級，不等於你的系統自動升級。  
清空舊時代的產物，讓出推論空間，才能真正享受新模型帶來的巨大紅利。